UNIVERSITAT JAUME I

Master's Degree in Intelligent Systems — Master's Thesis

Development and Implementation of a Person-Following System on a Mobile Robot — Part 2 Extension

Master's Thesis Report

Author: Juan Muñoz Rovira

Supervisor: Enric Cervera

Academic Year 2025–2026

Note on the status of this document

This report has seven chapters and is based on what is in the project repository and on the data from the lab sessions. The last session, on 23 September 2026, validated the “home” gesture (§7.8). Chapter 7 covers the tests up to the closing date, and whatever falls outside the scope of the thesis appears as future work (§7.10).

Table of Contents

Note on the status of this document 1
Table of Contents 2
Chapter 1 — Introduction and Problem Statement 4
1.1 Context and Motivation 4
1.2 Thesis Objectives 4
1.3 Scope and Limitations 5
1.4 Planning 5
1.5 Structure of the Report 5
Chapter 2 — System Architecture 7
2.1 Overview 7
2.2 Node and Topic Diagram 7
2.3 Description of Each Node 7
2.4 Coordinated Shutdown Management 9
2.5 Transform Tree (TF) 9
2.6 Configuration Parameters 10
2.7 ROS 2 Package Dependencies 10
2.8 Nav2 Architecture (Phase 3) 11
Chapter 3 — Use of Artificial Intelligence Tools 12
3.1 Declaration of Use 12
3.2 Tools Used 12
3.3 Reflection on the Ethical Use of Artificial Intelligence 12
3.4 Impact Estimate 12
Chapter 4 — Development Diary 14
Chapter 5 — State of the Art 14
5.1 Introduction and Scope of the Chapter 14
5.2 Person-Following Systems with Mobile Robots 14
5.3 Person Detection via 2D LiDAR (Leg Detection) 15
5.4 Vision-based Human Pose Estimation 15
5.5 LiDAR-camera Sensor Fusion for Robust Detection 15
5.6 Gesture-based Human–Robot Interaction 16
5.7 State Tracking: Bayesian Filters 16
5.8 Obstacle Avoidance and Motion Control 16
5.9 Mapping and Autonomous Navigation (SLAM and Nav2) 17
5.10 Synthesis: Positioning This Thesis Relative to the State of the Art 17
References 17
Chapter 6 — Implementation 19

6.1 Introduction and Scope 19
6.2 Package Structure and Configuration Management 19
6.3 detection_node — Clustering, Leg Features, and Temporal Filtering 19
6.4 visual_detection_node — Dual Detection and Gesture Recognition 20
6.5 tracking_node — Kalman Filter, PD Control, and Obstacle Avoidance 20
6.6 control_node — Finite-State Machine and Teleoperation 21
6.7 collision_handling_node 22
6.8 user_interface_node 22
6.9 slam_node (experimental, disabled by default) 22
6.10 Deployment: Building and Syncing with the Robot 22
6.11 Code Not Registered in the Package 22
Chapter 7 — Results and Evaluation 23
7.1 Validation Methodology 23
7.2 Experimental Setup 23
7.3 Result 1 — Sensor Fusion, No Motion (25 June 2026) 23
7.4 Result 2 — Progression of In-motion Continuity Fixes (8 July 2026) 24
7.5 Result 3 — Per-scenario Repetitions with the System Stabilized (22 July 2026) 26
7.6 Result 4 — Obstacle Scenario: from First Contacts to Contact-free Avoidance (22–23 July 2026) 27
7.7 Result 5 — Nav2: Localization and Autonomous Navigation (27 July 2026) 27
7.8 Result 6 — “Home” Gesture: Integrating Tracking and Navigation (23 September 2026) 28
7.9 Limitations of the Current Results 28
7.10 Future Work 29

Chapter 1 — Introduction and Problem Statement

1.1 Context and Motivation

At UJI's lab there was already a TurtleBot 2 that followed a person with its LiDAR, but it did not know where it was or how to get back to a place.

This thesis builds on an earlier project that produced a person-following system on a TurtleBot 2 (Kobuki base), able to detect and follow a person using an RPLIDAR A2M8 LiDAR and clustering (DBSCAN) over the leg pattern.

The original system had several limitations, and this extension comes from them:

- **No mapping: no map of the environment, which prevented any planned navigation.**
- **No global localization: if the person was lost, the robot could not orient itself.**
- **Limited interaction: no clear mechanism for the user to start and stop tracking.**
- **No camera integration: visual detection was not part of the control loop.**

1.2 Thesis Objectives

General objective

To extend the person-following system into a mobile assistance platform capable of autonomous navigation, environment understanding, and communication with the user indoors.

Specific objectives

1. **Gestural interaction module. Starting and stopping tracking with gestures, using MediaPipe Pose (right hand raised to start, left hand to stop). After replacing the camera (Logitech C270 → SPCA2650) and adjusting the visibility threshold, both gestures triggered reliably in real tests. Objective completed.**
2. **SLAM-based mapping. SLAM Toolbox was integrated with the LiDAR to build real-time maps of the lab, reused for localization and navigation. Objective completed.**
3. **Autonomous navigation with Nav2. Scope: AMCL over a saved map and navigation to a target point. The DDS latency failures were fixed by using longer commands, and navigation reached six of seven goals, including two ~8 m routes and avoidance of an unmapped obstacle. Objective completed.**
4. **LiDAR-camera sensor fusion. The camera supplies the bearing when the LiDAR finds no valid leg pair. The static validation gave 100% detection, but in motion jumps and angular saturation appeared; they were fixed with a continuity filter, a Mahalanobis gate and an angular speed limit, plus a corrected inverted sign in the PD control and the avoidance sector. Objective completed.**
5. **User guidance. Having the robot escort the person to predefined destinations in the building. First step validated on the real robot on 23 September 2026: with the "home" gesture, tracking and navigation work together in one system, with 9 of 11 home-return attempts completed (82%) and a 0.27 m mean arrival error (§7.8). Guidance to multiple destinations remains as future work (§7.10).**

6. **Exploratory objective. Try QR codes so the robot can recognize rooms. Still pending at the close of this chapter.**

1.3 Scope and Limitations

Development is carried out on the hardware available in the robotics lab at Universitat Jaume I:

- **Robot: TurtleBot 2 on a Kobuki base.**
- **Onboard computing unit: Intel NUC with Ubuntu 24.04 LTS and ROS 2 Jazzy.**
- **Sensors: 2D LiDAR RPLIDAR A2M8 (12 m range, 10 Hz) and an RGB camera (Logitech C270 until replaced by an SPCA2650, prompted by gesture framing issues). An Orbbec Astra RGB-D camera is also an option, but its integration is still pending.**
- **Test environment: indoor facilities at Universitat Jaume I.**

The following limitations are assumed:

- Navigation restricted to two dimensions (single floor).
- Person detection limited to the horizontal plane, at leg height, since that is the LiDAR's scanning plane.
- Communication between the control PC and the robot over the lab's WiFi network.

1.4 Planning

Development was organized into five phases, shaped by lab availability (limited sessions in June-July, none in August). Table 1.1 summarizes the status of each phase.

Table 1.1
Project phases

Phase	Description	Period	Status
1	Definition, review of the prior system, and initial drafting	May 2026	Completed
2	Gestural interaction module, SLAM integration, and sensor fusion	June 2026	Completed
3	Full autonomous navigation with Nav2	July 2026	Completed
4	Experimental validation at UJI's facilities	June–July 2026	Completed
5	Closing, final report, and defence	August–September 2026	Ongoing

Validation tasks not completed within the available lab schedule are documented as future work in Chapter 7.

1.5 Structure of the Report

The rest of the report is organized as follows:

- **Chapter 2 — System Architecture: nodes, topics and data flow.**
- **Chapter 3 — AI Tools: declaration of their use as development support.**

- **Chapter 4 — Development Diary: chronological log of technical progress.**
- **Chapter 5 — State of the Art: literature review connected to design decisions.**
- **Chapter 6 — Implementation: algorithms, control formulas and package structure.**
- **Chapter 7 — Results and Evaluation: analysis of the experimental validation data.**

Chapter 2 — System Architecture

2.1 Overview

The system is organized as independent ROS 2 nodes, communicating only over the message bus (DDS). This way nodes can be switched on or off, a module (for example the person detector) can be swapped without rewriting the rest, and only part of the system can be launched during development.

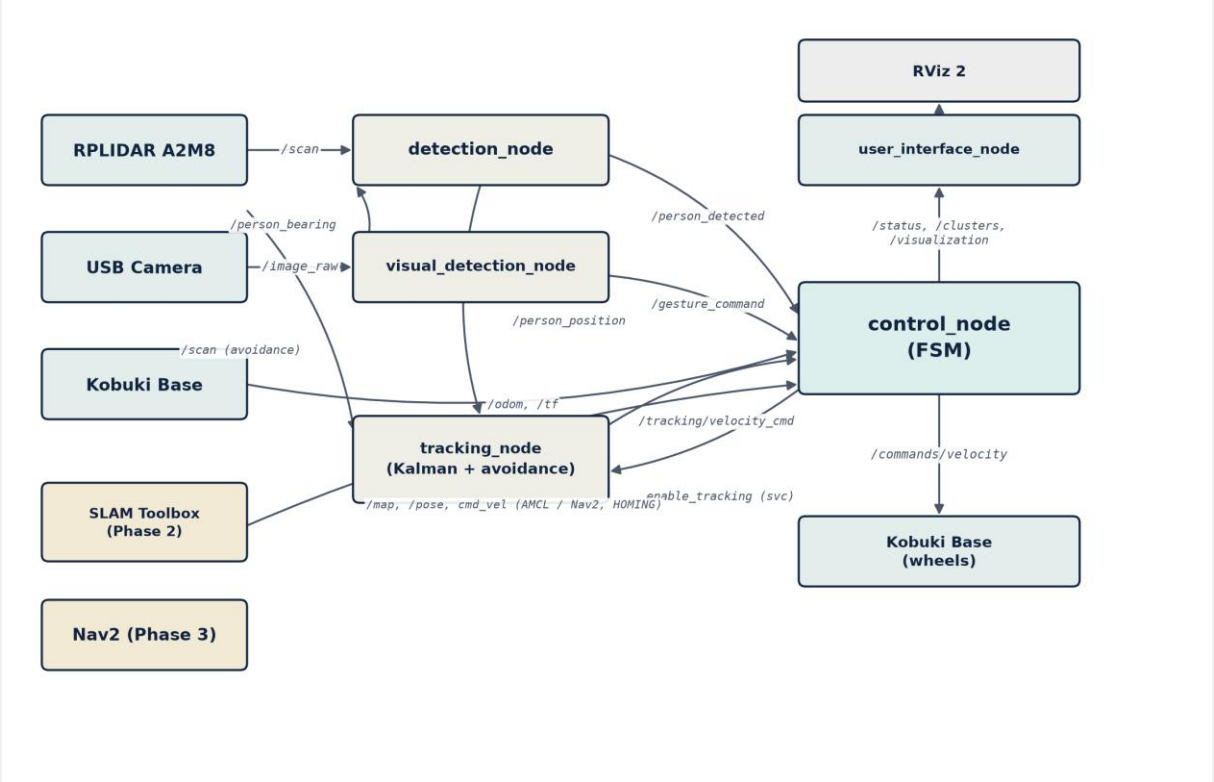
The whole system is packaged under the ROS 2 package person_follower, of type ament_python.

2.2 Node and Topic Diagram

Figure 2.1 summarizes the data flow between the main nodes. Since Phase 2, visual_detection_node also publishes /person_bearing to detection_node: the sensor-fusion fallback of §2.3.1, active only when the LiDAR finds no valid leg pair.

Figure 2.1

Node and topic flow of the person_follower system.



Note. Own elaboration from the project's source code.

2.3 Description of Each Node

2.3.1 detection_node

Function: person detection using the LiDAR.

The algorithm filters LiDAR noise by median, interpolates to increase angular resolution, and clusters points with DBSCAN (eps = 0.12 m, min_samples = 4). It then keeps the clusters that could be a leg (by size, radius and shape) and looks for pairs at a plausible distance.

As a fallback (Phase 2), without a valid leg pair the node uses visual_detection_node's bearing (MediaPipe) to pick the best-aligned LiDAR cluster and publishes its centroid. Without it, tracking stopped after 30 s without legs even if the camera saw the person. Validated without motion: 100% detection, bearing deviation ≈6°.

DBSCAN is reimplemented over scipy.spatial.cKDTree, and not over scikit-learn, because the NUC's Python version was not compatible with that library's compiled extension.

Configurable parameters: maximum detection distance (2.5 m, reduced from 6.0 m), DBSCAN neighborhood radius (0.12 m), leg cluster size (4-200 points) and camera timeout (1.0 s). It subscribes to /scan, /person_detected_visual and /person_bearing; publishes /person_detected, /person_position, /clusters/general and /clusters/legs.

2.3.2 visual_detection_node

Function: person detection and gesture recognition via camera.

The node processes each frame with MediaPipe Pose to detect the human skeleton, instead of the original HOG detector, which needed the full body in frame, something impossible at the real tracking distance (~1 m).

Gestures are recognized from MediaPipe Pose body landmarks (not Hands): right hand raised above the shoulder starts tracking, left hand stops it. Each gesture requires several consecutive confirmation frames and a wait time between commands, to reduce false positives.

The node also computes the person's horizontal bearing from the midpoint between the shoulders, based on the camera's field of view, and publishes it for detection_node's fusion. It subscribes to /image_raw and publishes /person_detected_visual, /gesture_command, /person_bearing, /pose/keypoints and /camera/image_processed.

2.3.3 tracking_node

Function: estimating the person's position and computing the velocity commands.

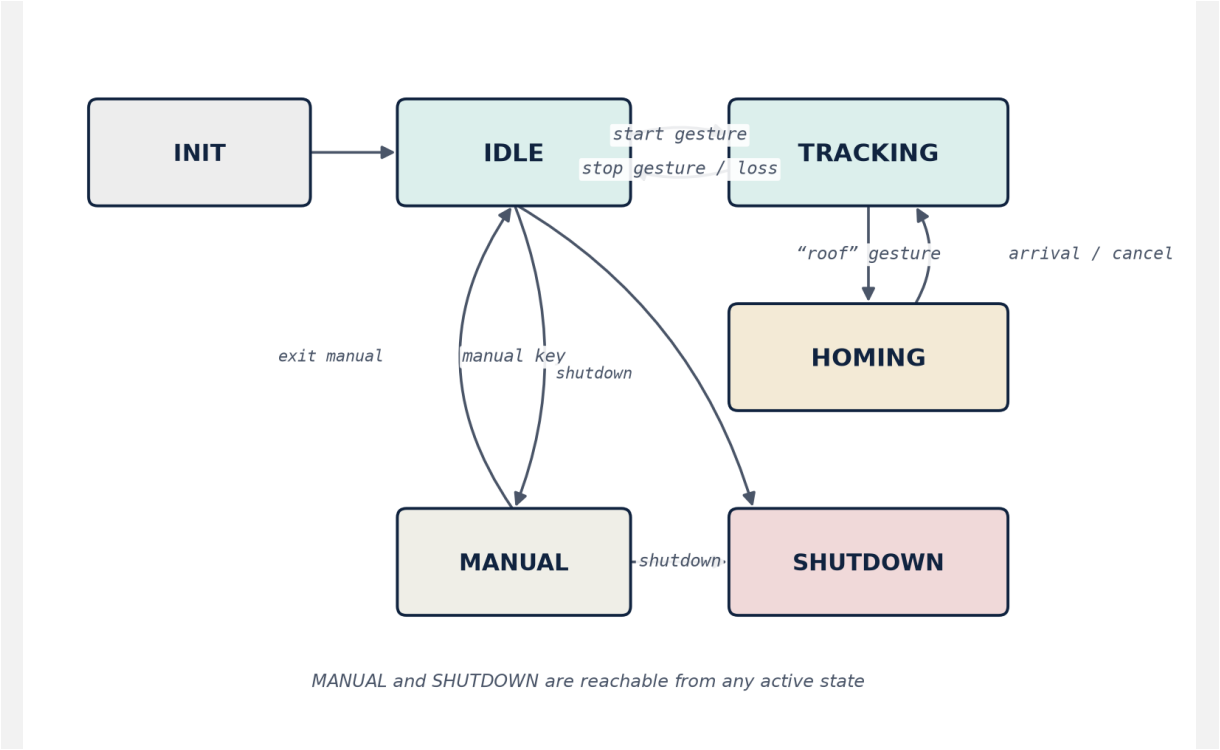
Applies a Kalman filter ([x, y, vx, vy, ax, ay], constant acceleration, §6.5) over detection_node's position, computing distance and angle to the person. Implements obstacle avoidance over the LiDAR's ±50° frontal sector: obstacles under 0.40 m trigger angular repulsion and reduce linear speed, and the robot stops if the person disappears for more than two seconds. Activated by the enable_tracking service, invoked by control_node.

2.3.4 control_node

Function: the system's central finite-state machine.

Implements the state machine of Figure 2.2: IDLE (waiting for an authorized person), TRACKING (relays tracking_node's velocities), HOMING (relays Nav2's velocity toward the “home” pose after the “roof” gesture, §7.8), MANUAL (keyboard teleoperation) and SHUTDOWN (coordinated shutdown). With the camera enabled, starting tracking requires the gesture; otherwise it starts automatically.

Figure 2.2
control_node's finite-state machine.



Note. Own elaboration from the project's source code.

2.3.5 collision_handling_node

Monitors the minimum distance to the nearest obstacle and publishes a collision-detected signal when any LiDAR scan point is under 0.4 m. control_node does not yet react to this signal; its integration is planned alongside Nav2.

2.3.6 user_interface_node

Takes care of real-time visualization: launches RViz 2 with the package configuration, publishes markers (estimated person position, LiDAR clusters, robot marker) and a periodic diagnostic status.

2.3.7 slam_node (experimental)

A basic 2D SLAM that we wrote ourselves in Phase 1. In Phase 2 SLAM Toolbox replaced it, as it is more robust, closes loops and can save maps; the node stays disabled by default.

2.4 Coordinated Shutdown Management

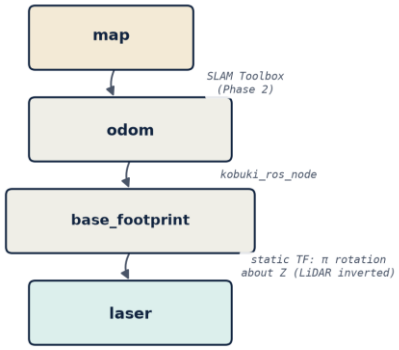
The system implements a coordinated shutdown to prevent the robot from being left with active velocities: control_node publishes a shutdown signal, subscribed nodes stop their activity and confirm the stop, and control_node waits for all confirmations before finishing.

2.5 Transform Tree (TF)

Figure 2.3 shows the transform tree. The base_footprint→laser transform is published statically, with a 180° rotation about Z, because the LiDAR is mounted inverted.

Figure 2.3

The system's transform tree (TF).



Note. Own elaboration from the project's source code.

2.6 Configuration Parameters

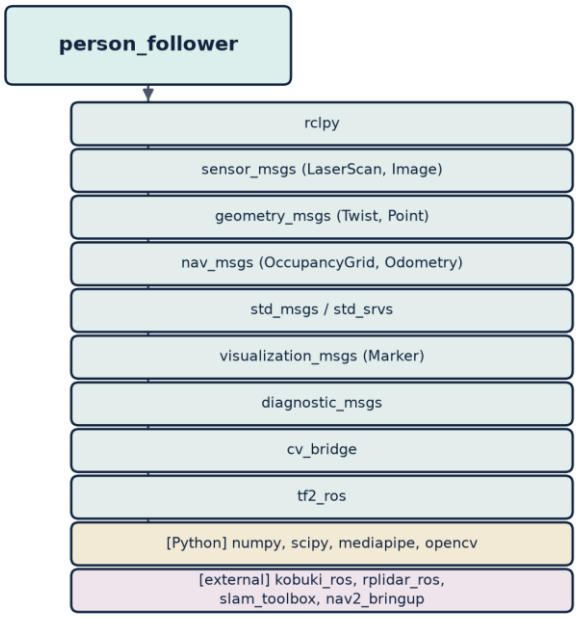
All tunable parameters are in a common configuration file, so detection thresholds, safety distances and robot behaviors can be changed without recompiling. SLAM Toolbox uses a separate configuration file.

2.7 ROS 2 Package Dependencies

Figure 2.4 summarizes the dependencies of the person_follower package, both internal to the ROS 2 ecosystem and external.

Figure 2.4

person_follower package dependencies.



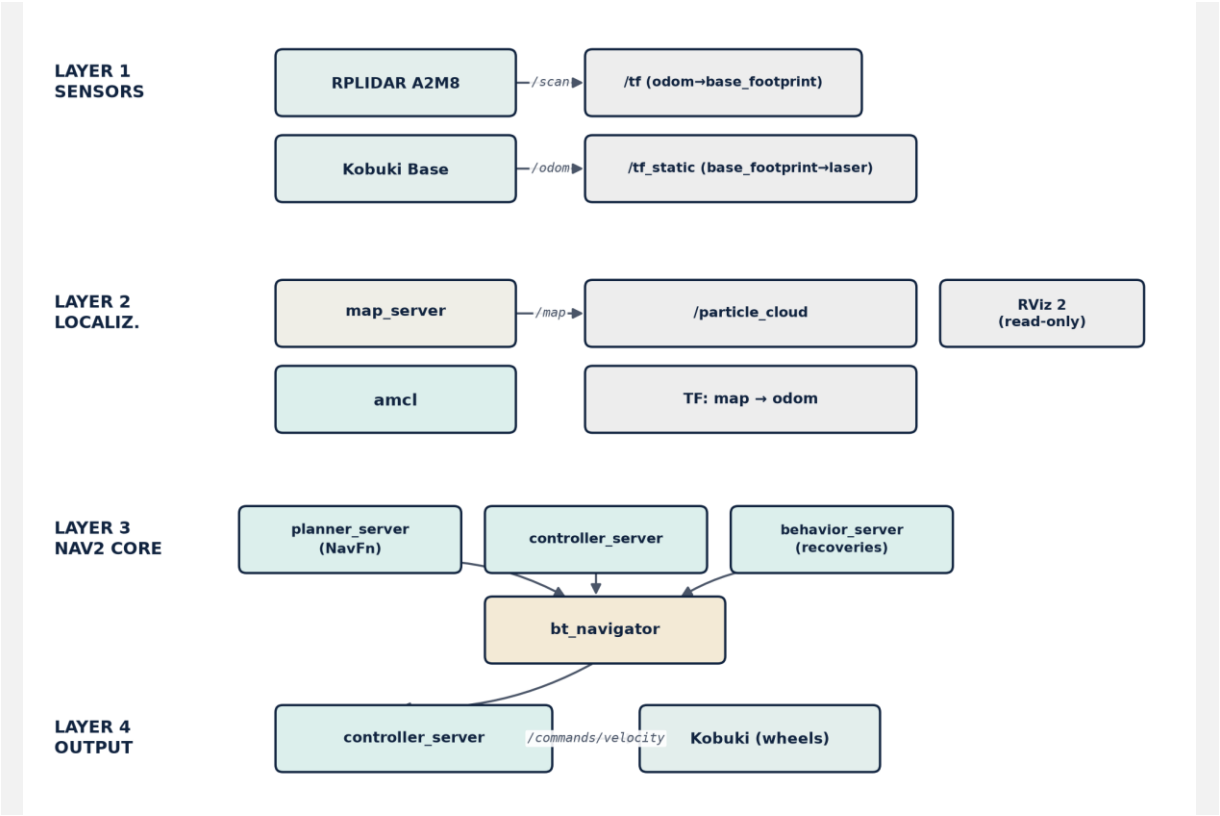
Note. Own elaboration from the project's source code.

2.8 Nav2 Architecture (Phase 3)

Autonomous navigation is organized in four layers (Figure 2.5): sensors (LiDAR and odometry); localization, where map_server publishes the map and AMCL combines scan, odometry and map to estimate the pose (map-odom TF); the Nav2 core (planner_server, controller_server, behavior_server and bt_navigator, coordinated by a behavior tree); and the output, where controller_server drives the wheels.

Figure 2.5

Layered architecture of autonomous navigation with Nav2.



Note. Adapted from the Nav2 architecture described in Macenski et al. (2020).

This architecture was validated in stages: a first test confirmed sensors and localization, and a later one also covered the Nav2 core and the output. That test reached six of seven navigation goals, including avoidance of an unmapped real obstacle.

Chapter 3 — Use of Artificial Intelligence Tools

3.1 Declaration of Use

This thesis uses artificial intelligence tools as technical support, much as one would use an advanced search engine or an automated documentation tool.

Many universities already accept that these tools help with learning and productivity, provided the student retains intellectual authorship: understanding the problem, design decisions, experimental validation and interpretation of results.

3.2 Tools Used

3.2.1 Conversational AI assistant (Claude, Anthropic)

The main use was diagnosing ROS 2 errors: compilation errors (ament_cmake, find_packages()); log analysis to distinguish the LiDAR's and the Kobuki base's serial ports; queries about ROS 2 interfaces; and code review, identifying an unintended double subscription in the tracking node.

In addition, in the final writing phase the assistant was used to rephrase and trim paragraphs in several chapters of the report, in order to fit the page limit and improve readability. The author reviewed and validated every change. The design decisions (architecture, algorithms and parameters), the robot tests, the experimental data, their interpretation and the conclusions are the author's responsibility.

3.2.2 Code autocompletion tools

During development, IDE autocompletion (Visual Studio Code with Pylance) was used to speed up repetitive ROS 2 code, such as declaring parameters or creating publishers and subscribers.

3.2.3 Search engines and documentation

The official documentation for ROS 2, SLAM Toolbox, Nav2, and MediaPipe was routinely used as a technical reference during development.

3.3 Reflection on the Ethical Use of Artificial Intelligence

When the assistant suggested a change, I tested it on the robot before keeping it. Once it pointed out a double subscription in tracking_node that I had not seen; I understood it and fixed it myself. That is the rule I followed: include nothing I do not understand, declare the use, and answer for everything that is in the code.

3.4 Impact Estimate

Table 3.1 summarizes, for the project's main tasks, the proportion developed by the author and the type of support received from artificial intelligence tools.

Table 3.1

Work share and AI support by task

Task	% developed by the author	AI support
System architecture design	100%	None
DBSCAN algorithm for leg detection	100%	Debugging
Kalman filter in tracking_node	100%	None

Task	% developed by the author	AI support
control_node state machine	100%	None
Identifying package configuration errors	60%	Diagnosis
Physical robot bring-up	90%	Log diagnosis
Technical documentation (this chapter)	90%	Rephrasing and trimming of paragraphs
Experiments and validation	100%	None

Chapter 4 — Development Diary

This chapter tells the story of the project session by session, with its technical milestones and the fixes that came up along the way. Table 4.1 gives the full timeline. Chapter 6 describes how each module ended up; here the focus is the process, with its back and forth. Phase 4 closed with the validation of the “home” gesture on 23 September 2026 (see also §7.8 and §7.10).

Table 4.1
Development timeline.

Date	Phase	Milestone / fix
21 May	P1	First session: fixed install/build errors, LiDAR/Kobuki device conflicts, SLAM Toolbox not activated (lifecycle), and MediaPipe/scikit-learn access without internet. Six nodes operational.
27 May	P1	Second session: 4 fixes (ROS 2 API, residual signal, leg geometric filter); Kalman 4→6 states, Mahalanobis gate, angular PD control; C270 camera with HOG.
late May	P1	Fixed jerky motion (hysteresis, Kalman prediction); first lab map. Phase 1 completed.
4-17 Jun	P2	Detection continuity tuning; MediaPipe Pose replaces HOG (framing issue); start/stop gestures validated.
25 Jun	P2	Camera bearing as disambiguator without a leg pair; DBSCAN reimplemented over scipy. 100% detection, no losses. Phase 2 completed.
8-9 Jul	P3	First in-motion validation: jumps and angular saturation, fixed with continuity filter, Mahalanobis gate and angular limit (jumps: 1/8 → 1/140).
9-15 Jul	P3	Gesture fixed (SPCA2650 camera); fixed uncontrolled turning and inverted PD control sign, verified against real odometry.
16-21 Jul	P3	Chapters 5-6 written; fixed inverted avoidance sector; turn-detection gap reduced from 2-4 s to ~1.5 s.
22-23 Jul	P3	Two light collisions (LiDAR blind spot, mis-referenced threshold); fixed speed reduction factor and added go-around maneuver: from 4 contacts to contact-free avoidance.
27 Jul	P3	AMCL converges with longer commands (DDS latency); Nav2 reaches 6/7 goals, including avoidance of an unmapped obstacle. Phase 3 completed.
29 Jul	P3-P4	Demo video recorded; 2 open limitations confirmed: avoidance does not distinguish person from obstacle, position jump after prolonged occlusion.
Aug-Sep	P4-P5	Phase 4: closing the evaluation scenario and consolidating metrics. Phase 5: final thesis review, defense prep, and repository tagging.

Chapter 5 — State of the Art

5.1 Introduction and Scope of the Chapter

This chapter puts the design decisions of Chapter 2 in context. It goes through the prior work in each area (2D LiDAR detection, vision-based pose, sensor fusion, gestures, Bayesian filters, obstacle avoidance and SLAM/Nav2), without trying to be an exhaustive review.

5.2 Person-Following Systems with Mobile Robots

Person-following is a classic service-robotics problem, combining perception, state estimation and motion control. With LiDAR alone, the geometry of the legs can be used and lighting is not a problem, but chair legs or columns get mistaken for legs. With a camera alone there is a lot of information (gestures, identity, orientation), but occlusions, poor light and a limited field of view cause trouble.

Fusion tries to keep the best of each: Sarmento et al. (2024) fuse time-of-flight with a monocular camera, and Kitamoto et al. (2025) propose tracking with 3D LiDAR and point-cloud projection.

These robots typically move indoors on differential bases, like the TurtleBot 2/Kobuki in this thesis. There, perception matters more than kinematics. On outdoor Ackermann bases (Li et al., 2026) maneuverability is limited, which adds constraints we do not have here.

This thesis's system falls into the multi-sensor fusion category (LiDAR + camera), with the clustering-based LiDAR detector inherited from the initial phase and a visual fallback path added in Phase 2 (§5.5).

5.3 Person Detection via 2D LiDAR (Leg Detection)

Detection from 2D scans at leg height is a well-established line: cluster segmentation, filtering by leg-compatible geometric features (size, radius, circularity, curvature), and pairing plausible candidates, optionally tracking their trajectory to distinguish gait from similar static objects.

Cha and Chung (2020) propose leg detection in a 3D feature space with 2D LiDAR for person-following robots; Yao et al. (2023) address differentiating multiple targets in multi-person scenarios. A recurring challenge is geometric ambiguity: chair/table legs, columns and other cylindrical objects can produce false positives, and partial occlusions —one leg blocking the other during a turn— break the pairing. Guerrero-Higueras et al. (2019) explore convolutional networks over raw 2D scans as an alternative to hand-crafted features, for robustness in cluttered environments.

This project's detection node (§2.3.1) follows the classic geometric-feature approach (DBSCAN clustering, filtering by size and radius, pairing by plausible distance) rather than a deep-learning alternative such as Guerrero-Higueras et al. (2019). That way no dedicated GPU is needed and the original system's already-validated detector is reused. The geometric ambiguity described above is what motivates the camera-based fallback of §5.5. The joint loss of LiDAR and camera observed during turns, still unresolved at the end of this chapter, is a concrete case of the partial-occlusion problem described by Cha and Chung (2020) and Yao et al. (2023).

5.4 Vision-based Human Pose Estimation

BlazePose (Bazarevsky et al., 2020) is a lightweight convolutional network. It produces 33 body keypoints at more than 30 fps without a dedicated GPU and is the basis of MediaPipe Pose, which is what we use here. Its GHUM Holistic extension (Grishchenko et al., 2022) adds 3D hand and face landmarks, though this thesis only needs shoulder and wrist landmarks for gestures and bearing.

Compared to HOG, used in the original version, landmark-based models do better in two ways. First, they do not need the full body in frame, which is impossible at the real tracking distance (~1 m). Second, they give meaningful anatomical points (wrist, shoulder) that can be used directly for gestures and orientation, instead of a plain bounding box. That is why HOG was replaced with MediaPipe Pose (§2.3.2).

5.5 LiDAR-camera Sensor Fusion for Robust Detection

LiDAR-camera fusion is an active area. LiDAR gives precise but ambiguous geometry; the camera gives semantics, but it suffers from occlusion and lighting and gives no reliable distance when monocular. The work I reviewed ranges from 360° vision-LiDAR (Li et al., 2026) to the transformer of Minase and Tanaka (2026) for LiDAR and thermal imagery. In both, fusion beats a single sensor.

This design uses a lighter scheme, without a depth camera: visual_detection_node computes an angular bearing from the shoulder midpoint (MediaPipe), and detection_node uses it only as a disambiguator when it finds no valid leg pair. This is an asymmetric late fusion: LiDAR remains the primary source of 2D position, and the camera only acts as a tiebreaker when the LiDAR geometry is not enough.

I made it asymmetric so as not to depend on calibrating the camera against the LiDAR precisely: I only use the horizontal field of view and a calibration sign. The cost is that the camera is wasted when the LiDAR already sees the legs, and during turns both lose the person at the same time, which is still unsolved.

5.6 Gesture-based Human–Robot Interaction

Gesture recognition has been reviewed extensively (Wang et al., 2022). The trend now is to use body landmarks rather than classifiers over the whole image. They are lighter and need no dataset: the gesture is just a geometric rule over points the model already provides. Work with MediaPipe reports 94-96% success in gesture-driven tasks (Zhang et al., 2025).

The system follows this rule-based approach. It has two gestures, right hand to start and left hand to stop, defined as geometric conditions over MediaPipe Pose landmarks, with frame confirmation and inter-command waiting (§2.3.2). The vocabulary is minimal on purpose, since interaction is limited to starting and stopping tracking.

5.7 State Tracking: Bayesian Filters

Bayesian filters (Kalman, EKF, UKF and particle filters) have been the standard way to track people with mobile robots for over two decades (Urrea & Agramonte, 2021). Bellotto and Hu (2010) show that the UKF can give results close to a particle filter at a much lower computational cost. That matters on embedded platforms like this project's, with no dedicated GPU and a CPU shared between LiDAR detection, MediaPipe and the rest of the ROS 2 nodes. Schulz et al. (2003) address simultaneous tracking of several people from a mobile robot with sample-based joint probabilistic data association filters.

The tracking node uses a plain linear Kalman filter with state [x, y, vx, vy, ax, ay] and a constant-acceleration model (§2.3.3, §6.5). It is the lightest option and it fits, because the motion and observation models are already linear: an EKF, a UKF or a particle filter would cost more and would only pay off if the observation model were non-linear.

5.8 Obstacle Avoidance and Motion Control

To turn the estimated position of a person into safe velocity commands, the classic method in the reactive navigation literature is the Dynamic Window Approach (DWA; Fox et al., 1997), which restricts the search to the robot's directly reachable velocity space, simulating short-term trajectories and selecting the one that avoids collisions. It is among the most widely adopted reactive avoidance methods indoors, and the one Nav2 (§5.9) uses as a standard local controller.

The tracking node avoids obstacles with a similar idea: it turns away and slows down when something is close (§2.3.3). It is a simplified version of full DWA, because it does not simulate trajectories and just applies a direct reactive rule over the frontal sector. Nav2 reserves full DWA for point-to-point navigation without active tracking.

5.9 Mapping and Autonomous Navigation (SLAM and Nav2)

For mapping we use SLAM Toolbox (Macenski & Jambrecic, 2021), the reference implementation for ROS 2 (synchronous and asynchronous mapping, localization on a saved map, map merging). In Phase 2 it replaced our own basic SLAM from Phase 1 (§2.3.7), because it is more robust and closes loops.

For navigation we use Nav2, the successor to the ROS 1 stack (Macenski et al., 2020), which separates global planning, local control and perception and coordinates them with behavior trees and lifecycle nodes (more in Macenski et al., 2023). This thesis only reaches a minimal demonstration: AMCL on a saved map and navigation to one goal. Nav2 can do much more, but the number of sessions was limited (§1.4).

5.10 Synthesis: Positioning This Thesis Relative to the State of the Art

None of the pieces is new. All the techniques already existed; the work was to make them run together on the NUC, which is short of computing power. The clearest decisions of our own are these:

- DBSCAN reimplemented over scipy.cKDTree, because scikit-learn was not compatible with the NUC's Python version (§2.3.1).
- A LiDAR-camera fusion scheme that is asymmetric on purpose and needs little calibration, unlike the more symmetric or deep-learning-based schemes in the recent literature (§5.5).
- A linear Kalman filter, not an EKF or UKF: since this system's observation model is already linear, nothing more is needed (§5.7).

I have not invented any algorithm. What the thesis contributes is the part of control_node that decides who commands the base, tracking or Nav2, so that they do not step on each other. That explains the decisions in Chapter 6 and the results in Chapter 7, including bringing tracking and navigation together in the “home” gesture (§7.8).

References

Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). *BlazePose: On-device real-time body pose tracking* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2006.10204

Bellotto, N., & Hu, H. (2010). Computationally efficient solutions for tracking people with a mobile robot: An experimental evaluation of Bayesian filters. *Autonomous Robots*, *28*(4), 425–438. https://doi.org/10.1007/s10514-009-9167-2

Cha, D., & Chung, W. (2020). Human-leg detection in 3D feature space for a person-following mobile robot using 2D LiDARs. *International Journal of Precision Engineering and Manufacturing*, *21*(7), 1299–1307. https://doi.org/10.1007/s12541-020-00343-7

Fox, D., Burgard, W., & Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, *4*(1), 23–33. https://doi.org/10.1109/100.580977

Grishchenko, I., Bazarevsky, V., Zanfir, A., Bazavan, E. G., Zanfir, M., Yee, R., Raveendran, K., Zhdanovich, M., Grundmann, M., & Sminchisescu, C. (2022). *BlazePose GHUM holistic: Real-time 3D human landmarks and pose estimation* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2206.11678

Guerrero-Higueras, Á. M., Álvarez-Aparicio, C., Calvo Olivera, M. C., Rodríguez-Lera, F. J., Fernández-Llamas, C., Martín Rico, F., & Matellán, V. (2019). Tracking people in a mobile robot from 2D LIDAR scans using full convolutional neural networks for security in cluttered environments. *Frontiers in Neurorobotics*, *12*, Article 85. https://doi.org/10.3389/fnbot.2018.00085

Kitamoto, S., Hiroi, Y., Miyawaki, K., & Ito, A. (2025). Robust human tracking using a 3D LiDAR and point cloud projection for human-following robots. *Sensors*, *25*(6), Article 1754. https://doi.org/10.3390/s25061754

Li, Z., Kou, D., Wang, Y., Li, W., Dong, Z., & Zhang, L. (2026). Auto-Follower: A person-following system for urban Ackermann human–machine collaborative robotics. *ACM Transactions on Autonomous and Adaptive Systems*, Article 3787977. https://doi.org/10.1145/3787977

Macenski, S., & Jambrecic, I. (2021). SLAM Toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, *6*(61), Article 2783. https://doi.org/10.21105/joss.02783

Macenski, S., Martín, F., White, R., & Clavero, J. G. (2020). The Marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2718–2725). https://doi.org/10.1109/IROS45743.2020.9341207

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., & Ferguson, M. (2023). From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2. *Robotics and Autonomous Systems*, *168*, Article 104493. https://doi.org/10.1016/j.robot.2023.104493

Minase, Y., & Tanaka, K. (2026). *A dual-stream transformer architecture for illumination-invariant TIR-LiDAR person tracking* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2604.00363

Sarmento, J., Neves dos Santos, F., Silva Aguiar, A., Filipe, V., & Valente, A. (2024). Fusion of time-of-flight based sensors with monocular cameras for a robotic person follower. *Journal of Intelligent & Robotic Systems*, *110*(1), Article 30. https://doi.org/10.1007/s10846-023-02037-4

Schulz, D., Burgard, W., Fox, D., & Cremers, A. B. (2003). People tracking with mobile robots using sample-based joint probabilistic data association filters. *The International Journal of Robotics Research*, *22*(2), 99–116. https://doi.org/10.1177/0278364903022002002

Urrea, C., & Agramonte, R. (2021). Kalman filter: Historical overview and review of its use in robotics 60 years after its creation. *Journal of Sensors*, *2021*, Article 9674015. https://doi.org/10.1155/2021/9674015

Wang, X., Shen, H., Yu, H., Guo, J., & Wei, X. (2022). Hand and arm gesture-based human-robot interaction: A review. In *Proceedings of the 6th International Conference on Algorithms, Computing and Systems* (pp. 1–7). Association for Computing Machinery. https://doi.org/10.1145/3564982.3564996

Yao, H., Peng, J., Liao, Z., Zhao, R., & Dai, H. (2023). Leg detection for socially assistive robots: Differentiating multiple targets with 2D LiDAR. In F. Sun, Q. Meng, Z. Fu, & B. Fang (Eds.), *Cognitive systems and information processing* (Communications in Computer and Information Science, pp. 87–103). Springer Nature Singapore. https://doi.org/10.1007/978-981-99-8018-5_7

Zhang, G., Su, J., Zhang, S., Qi, J., Hou, Z., & Lin, Q. (2025). Research on deep learning-based human–robot static/dynamic gesture-driven control framework. *Sensors*, *25*(23), Article 7203. https://doi.org/10.3390/s25237203

Chapter 6 — Implementation

6.1 Introduction and Scope

Chapter 2 shows the layout of nodes and topics. Here I go into the code: how each node works inside and why I left it that way.

Not all code files are running: the install script registers seven nodes as executables (§6.2), and several directories keep unregistered earlier versions. Only the code that actually runs is described here; §6.11 explains the legacy code.

6.2 Package Structure and Configuration Management

The package is of type ament_python, with one directory per node, and registers seven executables: control_node, visual_detection_node, detection_node, tracking_node, user_interface_node, collision_handling_node, and slam_node.

Each node's adjustable parameters are in a single configuration file, with one section per node, which the launch files load, so thresholds can be changed without recompiling. Each parameter has a comment justifying it and, often, the date and reason for its last change.

The main launch file starts five nodes: control_node, tracking_node, visual_detection_node, detection_node and user_interface_node. collision_handling_node and slam_node are left out: the former because control_node does not yet react to its signal (§2.3.5), and the latter because it is disabled (replaced by SLAM Toolbox, Phase 2).

6.3 detection_node — Clustering, Leg Features, and Temporal Filtering

In-house DBSCAN on a Nearest-Neighbour Tree

This node's DBSCAN is reimplemented over scipy's cKDTree and not over scikit-learn, because it was not compatible with the NUC's Python version (§2.3.1). It follows the usual semantics: a point is a core point if it has enough neighbors within a radius, with radius-based neighbor queries and breadth-first expansion. The computational cost is negligible for the hundreds of points in a filtered scan. The neighborhood radius (0.12 m) and the minimum number of neighbors (4) were calibrated to the RPLIDAR A2M8's real angular resolution.

Multi-feature Geometric Leg Filter

The node extracts eight features per cluster (size, radius, spread, width, height, aspect ratio, circularity, compactness) and applies six sequential filters, combining the original inherited filter with additional features to reduce false positives (furniture legs) and false negatives (occluded legs). Two thresholds were corrected after confirming they rejected real legs.

Leg pairing is a greedy nearest-neighbor algorithm: for each unpaired cluster, the closest one within a coherent range (0.05-0.35 m) is sought and combined into a candidate whose centroid is the estimated position. No global optimization (Hungarian assignment) is used, given the low number of clusters per scan.

Temporal Continuity and Confirmation Filtering

Three temporal filtering mechanisms guard against false positives: temporal persistence (a detection must hold for several scans to be confirmed, and fail as many to be lost); a cumulative-drift filter, discarding candidates whose distance from the oldest confirmed position exceeds the plausible displacement; and a consistency confirmation, stricter for the camera fallback (always requires confirmation).

The fusion fallback: without a valid leg pair, the node computes the expected angle in the laser frame from the camera bearing, picks the cluster with the smallest angular deviation, and requires repetition over several scans before publishing it. This confirmation is independent from that of leg pairs, since a single cluster is a weaker signal.

The node flips the sign of the coordinates right before publishing, at the boundary between the laser's raw frame (front ≈π radians) and tracking_node's standard convention (front on the positive x axis). Keeping the rest of the logic in the raw frame and flipping only on publish avoids rewriting all the filtering, at the cost of reading internal variables in the raw frame.

6.4 visual_detection_node — Dual Detection and Gesture Recognition

The node supports two interchangeable detectors: MediaPipe Pose if available, or HOG as an automatic fallback (resolved via import exception handling). Without MediaPipe there are no landmarks, so gestures are unavailable in HOG mode. HOG, inherited from Phase 1 and replaced as primary by MediaPipe Pose (§2.3.2, §5.4), is kept as a fallback.

Video capture runs in a thread independent of the ROS 2 executor, discarding stale buffered frames and always processing the most recent one. Publishing is capped to a target rate, measuring actual processing time to adjust the wait.

Gesture recognition and bearing computation reuse the same MediaPipe Pose landmarks (shoulders, wrists, hips). The start gesture triggers when the right wrist rises above the right shoulder, and stop with the left; the decision margin scales with torso size, not fixed in pixels, and each gesture requires confirmation over several frames plus a wait between commands. The bearing is computed from the shoulder midpoint, published only if both exceed the minimum visibility.

6.5 tracking_node — Kalman Filter, PD Control, and Obstacle Avoidance

Along with detection_node, it is the node with the most documented iterations. The active version replaces an earlier, simpler design without an acceleration model, kept as legacy code (§6.11).

Six-state Kalman Filter with a Constant-Acceleration Model

The filter keeps a six-dimensional state (position, velocity, acceleration), versus four in the inherited design, with observation measuring only position. The transition matrix implements constant acceleration, with process noise correlated by powers of the time step; the correction uses Joseph's form, costlier but numerically more stable.

Mahalanobis Gate with Streak-based Confirmation

Each observation goes through a Mahalanobis gate against the 95% chi-squared threshold (2 d.o.f.): it is only rejected if it exceeds that threshold by several times, and if several consecutive observations fall outside it, the filter re-anchors to the latest one, assuming a real reappearance. An isolated

observation is ignored; a sustained streak is treated as a legitimate reappearance, the same confirmation pattern used by detection_node's continuity mechanisms (§6.3).

Angular Velocity Controller

Angular velocity is computed in five steps: (1) PD with a ±8° dead zone; (2) coupling with linear speed, reducing turning at high speed; (3) close-range attenuation, from zero to full value based on the reference distance; (4) soft start over the first second and a half; and (5) a per-cycle change limit, guarding against a large real error or a one-off detection jump.

Linear speed follows a non-linear ramp based on distance to the person, smoother than a linear relation near the tracking reference distance, avoiding abrupt starts when the person moves slightly away from that distance.

Reactive Obstacle Avoidance (not a Real Dynamic Window Approach)

The original documentation called this block Kalman + DWA, but tracking_node does not have a real DWA, which would simulate trajectories in velocity space (§5.8). That DWA exists in a separate file that is not registered (§6.11). The active mechanism is reactive, repulsion-field style: for each point in the ±50° frontal sector within a threshold (0.40 m, raised from 0.35 m), a weight is accumulated that gives more importance to closer, more frontal obstacles, and the final angular adjustment is the weighted average.

Linear-velocity reduction factor, fixed on 22 July. The original formula never brought the velocity down to zero, and its lower bound was dead code (§4.3, §7.6). So the robot never fully braked against a sustained obstacle, and there were two light contacts even though the obstacle was detected. The current formula uses the real minimum distance in the frontal sector: a ramp that is 1.0 at the threshold and drops to a genuine 0.0 (hard stop) at a second, closer threshold (0.25 m), leaving ~10 cm of margin.

Go-around manoeuvre, added on 22 July. If the reduction factor stays below a threshold (0.5) for 1.5 s sustained, the node executes a tight 1.5 s turn in the direction already indicated, followed by a short straight advance at low speed; it aborts and restarts the turn if the factor drops again. It is a way out of the deadlock instead of pushing forward indefinitely. It is a first version and has not been validated systematically yet (Chapter 7).

It is cheaper than a full DWA on purpose: it does not reason about kinematic constraints or look several steps ahead. That is enough for low-speed reactive avoidance, but it is a clear simplification compared with Nav2's DWA (§5.8-5.9). The architecture label has been corrected.

6.6 control_node — Finite-State Machine and Teleoperation

The state machine (INIT → IDLE ↔ TRACKING, with MANUAL and SHUTDOWN reachable from any active state) is a single class with the current state as a text string. Two mechanisms prevent erratic behavior: hysteresis on detection loss, waiting before returning to idle; and gesture-based authorization independent of detection (with the camera disabled, authorization is granted automatically).

Keyboard teleoperation runs on a separate thread reading input in unbuffered mode, to capture keys without blocking the ROS 2 executor, and disables itself if standard input is not an interactive terminal. This is needed because a leftover interrupt signal can remain in the buffer and be read as an unexpected signal when the system launches.

6.7 collision_handling_node

It is the simplest node: it applies a single distance threshold over the LiDAR scan's minimum, publishing the collision signal only on state changes. It is not included in the main launch and control_node does not yet subscribe to its signal (§6.2). It is infrastructure prepared for Nav2, which can react through its recovery behaviors.

6.8 user_interface_node

It implements no perception or control: it is a state aggregator. It subscribes to the status topics of the other nodes and republishes visualization markers for RViz 2 (estimated position, LiDAR clusters, robot marker), a text panel and a periodic diagnostic on mode changes. It also launches and closes RViz 2 in a coordinated way.

6.9 slam_node (experimental, disabled by default)

A minimal, custom 2D SLAM implementation: an in-memory occupancy grid, static TF between map and odometry, updated from the LiDAR and the person's position, without scan matching or loop closure: an occupancy accumulator, not real SLAM. SLAM Toolbox replaced it in Phase 2 (§5.9), and it is kept disabled as a historical reference.

6.10 Deployment: Building and Syncing with the Robot

The package is built on the NUC with symlinks to the source files, letting the executor pick up changes by relaunching the node, without recompiling after every edit. A sync script automates copying the most-modified files (detection, tracking, configuration); the rest requires manual copying, a real limitation to keep in mind for future extensions.

6.11 Code Not Registered in the Package

Several directories keep more than one complete node class, of which only one is registered: the control node keeps two earlier versions; the tracking node keeps a real DWA implementation (velocity-space search and RViz trajectory visualization), not running; and the user-interface node keeps an unregistered earlier version.

None of these files is imported by another module: it is not dead code, but earlier versions of the same class kept in the working tree. The real DWA matters because the documentation kept saying that tracking_node used it, when the active node uses the reactive controller of §6.5. That was confusing, and it is worth cleaning up before the final delivery.

Chapter 7 — Results and Evaluation

The data in this chapter comes from the tests at UJI's robotics lab, up to the final session on 23 September 2026 (§7.8). Several scenarios have few samples, which is explained in §7.9. What falls outside the scope of this thesis is in §7.10.

7.1 Validation Methodology

Each take is recorded on the NUC as a ROS 2 bag, converted to tabular format and analyzed externally, producing metrics and four figures (distance, angle and speed over time, and trajectory). Standard metrics: distance error (MAE/RMS), mean angular error, peak speeds, % detection and losses.

7.2 Experimental Setup

- **Robot: TurtleBot 2 on a Kobuki base, with an onboard compute unit running ROS 2 Jazzy.**
- **Sensors: RPLIDAR A2M8 LiDAR and an RGB camera. The camera changed midway through the validation campaign (§1.3): the no-motion fusion take (25/06) and the three takes from 08/07 (§7.4) were recorded with the original Logitech C270; all takes from 09/07 onward (§7.5, 7.6, and 7.7) were already recorded with the SPCA2650 that replaced it.**
- **Environment: UJI's robotics lab, indoors, tiled floor.**
- **Target tracking distance: 1.50 m in the 25 June take; 1.00 m from the 8 July takes onward, after a parameter adjustment between sessions.**
- **Tracking activation: via camera gesture under normal conditions; the 8 July takes used a manual activation mechanism because the gesture was not reliable with that session's camera framing.**

7.3 Result 1 — Sensor Fusion, No Motion (25 June 2026)

First validation of the LiDAR-camera fusion fallback mechanism, with the robot's base inhibited so the detection mechanism could be tested safely before validating it in motion.

Table 7.1
Fusion without motion, 25/06

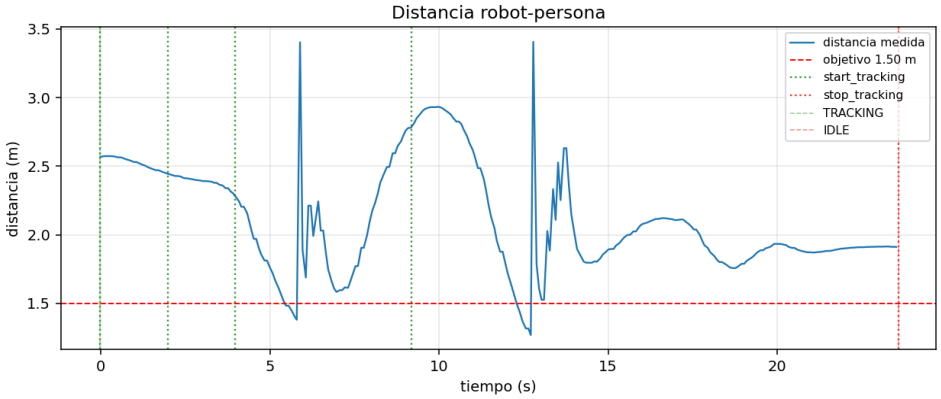
Metric	Value
Duration	23.5 s (285 samples)
Target distance	1.50 m
Distance error (MAE / RMS)	0.627 m / 0.733 m
Min / max distance	1.27 m / 3.41 m
% of time with person detected	100.0%
Number of detection losses	0
Camera bearing vs. chosen cluster deviation	≈ 6°

With the fusion mechanism active, detection was continuous throughout the whole take, even though the LiDAR alone could not reliably tell legs apart in this configuration, which was the original motivation for the fallback mechanism (§2.3.1, §5.5). The mean angular error is not representative in this take:

with the base inhibited, angular velocity never corrects orientation toward the person, so this figure measures the lack of motion rather than a detection failure.

Figure 7.1

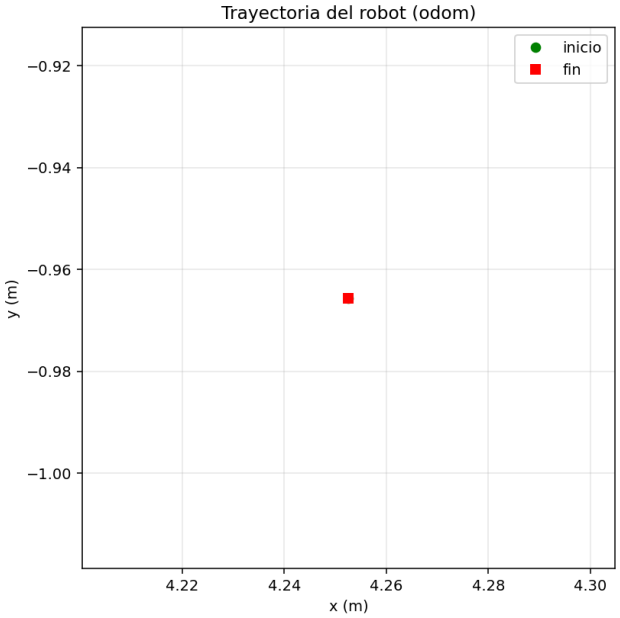
Robot-to-person distance over time, no-motion fusion take (25/06/2026).



Note. Own elaboration from the recorded data of the take.

Figure 7.2

Robot trajectory (odometry), no-motion fusion take. The base remained stationary by design of the test.



Note. Own elaboration from the recorded data of the take.

7.4 Result 2 — Progression of In-motion Continuity Fixes (8 July 2026)

First tracking test with the robot actually moving. The initial take revealed detection jumps and near-permanent angular saturation; the next two takes verify each fix separately (§4.3).

Table 7.2

Continuity corrections, 08/07

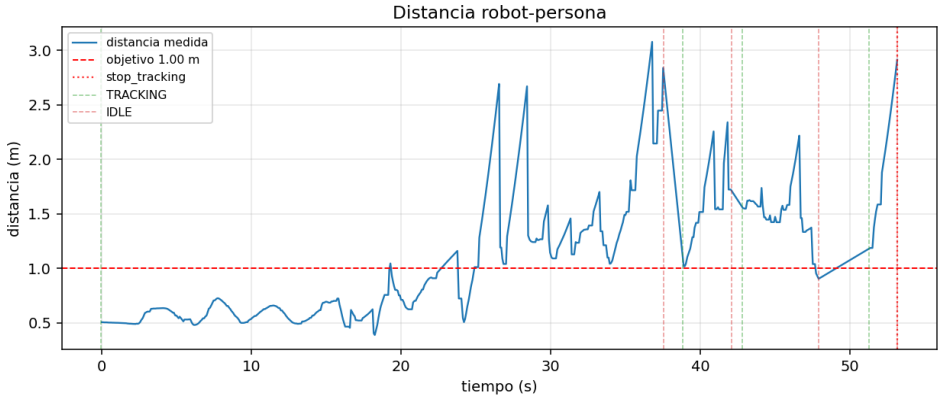
Take	Duration	% detection	Losses	MAE dist.	RMS dist.	Jumps >0.8 m	wz saturation
Original (unfixed)	759.8 s	32.3%	79	0.519 m	0.935 m	3.5%	95.7%
+ continuity filter	249.0 s	71.7%	43	0.534 m	0.725 m	2.2%	99.3%
+ Mahalanobis and wz limit	53.1 s	82.6%	13	0.491 m	0.609 m	0.7%	86.9%

The jump and saturation figures were recalculated on 21 July with the standard analysis pipeline over the three original logs; the jump percentage reproduced exactly in two of the three takes and came out somewhat lower in the third, a minor difference attributable to reconstructing a previously ad-hoc methodology.

The three chained fixes clearly reduced implausible position jumps and increased detection: fewer jumps yield a more stable Kalman filter. Angular saturation, however, does not improve consistently, because the fixes target detection continuity and not the angular controller's gain. The fix that does reduce it (proximity attenuation, §6.5) was added later, on July 15.

Figure 7.3

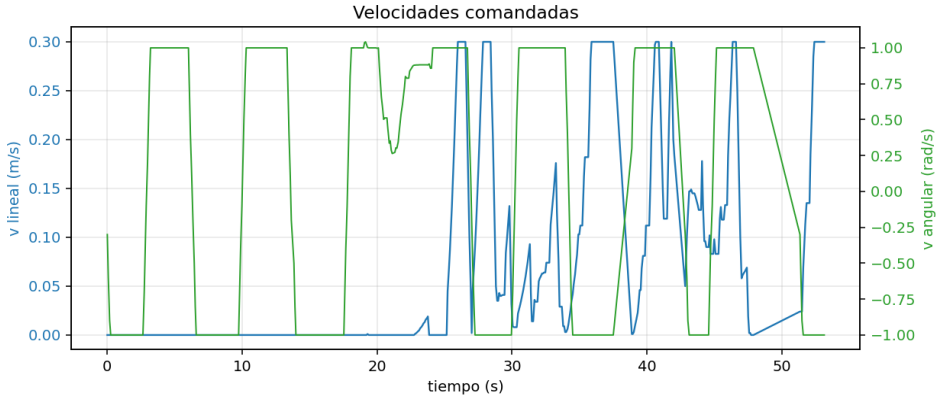
Robot-to-person distance over time, take with all three continuity fixes applied.



Note. Own elaboration from the recorded data of the take.

Figure 7.4

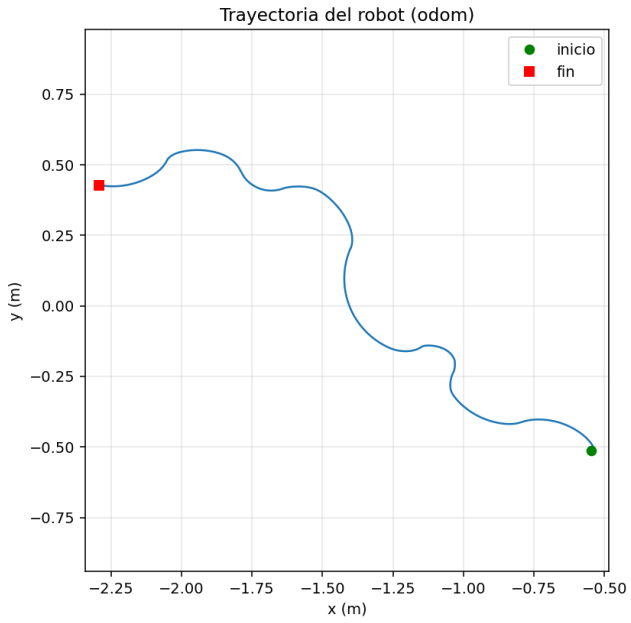
Linear and angular velocity over time, same take.



Note. Own elaboration from the recorded data of the take.

Figure 7.5

Robot trajectory (odometry), same take.



Note. Own elaboration from the recorded data of the take.

7.5 Result 3 — Per-scenario Repetitions with the System Stabilized (22 July 2026)

With the July 21 fixes already in production, two to three takes were repeated for five collision-free validation scenarios: straight line, curve, short distance, stop, and occlusion.

Table 7.3

Scenarios with the stabilized system, 22/07

Scenario	Duration	MAE dist.	Mean ang.	% detec.	Losses	Jumps >0.8 m	Stable sat.
Straight (contaminated)	41.4 s	0.626 m	62.0°	76.2%	7	6.8%	13.8%
Straight (clean)	25.9 s	0.420 m	26.5°	100%	0	2.3%	1.8%
Curve #1	33.5 s	0.794 m	14.5°	100%	0	2.8%	6.8%
Curve #2 (short)	12.2 s	0.759 m	23.7°	97.1%	1	2.0%	20.8%
Curve #3	14.3 s	0.976 m	6.2°	100%	0	0.9%	0.0%
Stop	36.5 s	0.326 m	5.9°	100%	0	0.0%	0.0%
Short #1 (min. 0.15 m)	32.5 s	0.443 m	32.8°	100%	0	2.0%	25.1%
Short #2 (short, min. 0.42 m)	8.4 s	0.628 m	4.9°	98.8%	1	0.9%	0.0%
Short #3 (min. 0.02 m)	18.7 s	0.508 m	46.8°	100%	0	1.2%	28.6%
Occlusion	30.6 s	0.678 m	16.2°	100%	0	2.5%	1.5%

The first straight-line take was contaminated by a residual tracking state (the oscillation already documented on June 17); it was not repeated after de-authorizing tracking before each take. The short durations of curve #2 and short #2 stem from the gesture taking time to arrive; short #3 reached 0.02

m, with expected saturation. Occlusion caused no real loss, thanks to the Kalman filter and the single-leg fallback.

7.6 Result 4 — Obstacle Scenario: from First Contacts to Contact-free Avoidance (22–23 July 2026)

After the two real contacts described in §4.3 (21 July session), the obstacle scenario was retried with the safety threshold increased. Seven takes were carried out over two sessions, with two intermediate code changes.

Table 7.4
Obstacle, 22–23/07

Take	Configuration	Result
Obstacle v3 (furniture)	Safety threshold raised, linear reduction factor unfixed	Light contact — the reduction factor never reached zero, the robot "dragged" without fully braking
Obstacle v4 (furniture)	Same	Light contact again, same pattern
Obstacle v5 (furniture)	Reduction factor fixed: real ramp to zero between the warning and stop thresholds	No contact — much briefer encounter
Obstacle v6 (furniture + chair)	Same	No contact — the chair did not trigger avoidance in this take
Obstacle v7 (furniture + chair)	+ go-around manoeuvre (tight turn and short advance)	No contact — manoeuvre triggered four times, completed successfully in all cases except one correct safety abort
Obstacle v8 (furniture + chair)	Same	Furniture correctly avoided; direct contact with the chair, not detected at all by the LiDAR
Obstacle v9 (furniture, 23/07)	No code changes from v7/v8	No contact — second clean repetition of the fix, confirming the result in a different session

The fault was in the formula: the reduction factor never dropped below 0.4 because the lower bound never applied (dead code). The robot saw the obstacle and still did not fully stop, which is why it grazed it twice. The fix based the factor on the actual minimum distance in the frontal sector, with a hard stop past a second threshold, and the go-around maneuver was added.

In take v8 the robot hit the chair, the fifth contact so far. We had feared it: the LiDAR scans at about 47 cm and does not see the thin legs of a chair. We did not repeat the test until we had another sensor, either the RGB-D camera we have or a second LiDAR at a different height.

7.7 Result 5 — Nav2: Localization and Autonomous Navigation (27 July 2026)

First real test of the complete Nav2 stack (specific objective 3). AMCL localization seemed to freeze after its first update. The problem was that the very short test commands were not arriving in time because of node-discovery latency; with longer commands it converged without trouble.

The full navigation phase was then tested for the first time, with seven goals sent from the RViz interface:

Table 7.5
Nav2 navigation, 27/07

Goal	Approx. distance	Result
1	≈ 3.7 m	Success
2	≈ 5.9 m	Success
3 (real unmapped obstacle)	≈ 5.3 m	Success — detected and avoided by the local costmap
4	≈ 8.0 m	Success
5	≈ 8.1 m	Success
6 (goal change mid-path)	≈ 0.6 m	Failure — automatically recovered by the lifecycle manager in about 2 s
7 (after recovery)	≈ 3.6 m	Success

Six of seven goals were completed (86%), including two long ~8 m routes and one with avoidance of a real obstacle not present in the map, dodged via the local costmap. The single failure self-recovered through Nav2's lifecycle manager, without recurring. During the session we noticed that the saved map did not match the real space well. We recorded it again and validated it before replacing it, so all the results are already with the new map.

7.8 Result 6 — "Home" Gesture: Integrating Tracking and Navigation (23 September 2026)

Final lab session. The person makes the "roof" gesture (both hands joined above the head) and the robot stops following. It then returns on its own, via Nav2, to a predefined "home" pose. This is the first time tracking and navigation work in a single system. control_node decides which velocity source reaches the base (§2.3.4, §6.6).

Table 7.6
Metrics of the "home" gesture, 23/09

Metric	Value
Home-return attempts completed	9 of 11 (82%)
Time from gesture to home	15.5 s on average (11-21 s)
Arrival error	0.27 m / 8° on average (Nav2 tolerance: 0.25 m / 14°)
Cancellation with the left hand	Confirmed by Nav2 in 20 ms, ~7 cm of travel
Gesture false positives	0 in ~4 min of free gesturing
Roof gesture recognized from behind	2 of 2 (slower: the head hides one wrist)

The two failures were not caused by the gesture or the state machine. Localization (AMCL, σ≈0.5 m) placed the robot inside the safety zone of a mapped obstacle, and from there the planner found no valid route. The robot returned to rest safely. This is a first result: 11 attempts in a single session (see §7.9).

7.9 Limitations of the Current Results

- **Small sample.** Several rows come from a single take, so there is no measure of variability between equivalent tests. The stop and occlusion tests have only one repetition at the close of this chapter.
- **Durations and activation gesture.** The takes in §7.4 differ a lot in duration, so the figures are proportions within each take, not normalized values. Also, the 8 July takes were started with a manual mechanism and not with the real gesture, so objective 1 only appears in the takes after 9 July.
- **Nav2 has a single session, with no repetitions.** So we cannot tell whether what we see is a pattern or a one-off event. We also have not looked in depth at why the single failure happened.
- **Objective 5: only a first step, with a limited sample. The "home" gesture (§7.8) brings tracking and navigation together for the first time: 9 of 11 home-return attempts completed in a single session (11 attempts). The 2 failures came from localization (AMCL, ±0.5 m), which left the robot next to a mapped obstacle with no room for the planner to trace a route, and not from the gesture or the state machine. Guidance to multiple destinations, which would be the full scope of the objective, remains as future work (§7.10).**
- **Shared compute resources.**The NUC runs the LiDAR, the camera with MediaPipe, tracking, SLAM, Nav2 and the diagnostics interface all at once. In the early phases this caused disconnections and port conflicts, and the code had to be optimized considerably (duplicate ports, publication rates, a simpler detection pipeline; Chapter 4). With everything active, the final measured load is ~30% per core (4 cores), with no saturation. What we have not checked is whether a momentary spike contributed to the two localization failures.

7.10 Future Work

Four lines of future work remain, beyond the scope of this thesis: guiding the robot to several destinations (a gesture or a QR code per destination, which would complete objective 5); a departure manoeuvre when localization leaves the robot next to an obstacle, before requesting a route from Nav2; repeating the stop and occlusion tests with more samples; and testing the system with more people, environments and lighting conditions.